

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 7

Operating Systems

Topic: Three Small Programs

Processes, Redirection and a Pipe on Linux

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Short Theory **4**

1.1 fork, exec and wait 4

1.2 File Descriptors, Redirection and Pipes 4

2 fork + exec + wait **5**

2.1 The Program 5

2.2 Running It 5

3 A Zombie, on Purpose **7**

4 Redirection: ls -l > out.txt **9**

5 A Pipe: ls /etc | wc -l **11**

6 Optional: Three Stages **13**

Conclusions **15**

INTRODUCTION

The lecture asked three questions: how is a process born, where does `printf` write, and how does `ls | wc -l` work. The purpose of this laboratory work is to run the answers. Four short C programs reproduce, step by step, what the shell does: `run.c` starts a command with `fork`, `exec` and `wait`; `zombie.c` shows what happens when the parent does not wait; `redirect.c` implements `ls -l > out.txt`; and `pipeline.c` builds `ls /etc | wc -l` with one pipe and two children. I also wrote the optional `pipeline3.c` with three stages.

The work was done on my Ubuntu 24.04 machine, as the user `sergiu`:

```
$ mkdir -p ~/os-lab7 && cd ~/os-lab7
$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
```

Listing 1: Preparation

1 SHORT THEORY

1.1 fork, exec and wait

A new process in Unix is born in two steps. `fork()` creates an almost exact copy of the calling process: the same code, a copy of the memory and of the table of open file descriptors. It returns twice – 0 in the child and the PID of the child in the parent. `exec()` (here `execlp`, which also searches the `PATH`) replaces the program of the calling process with a new one; the PID and the open descriptors stay the same, and on success `exec` never returns. `wait()` / `waitpid()` blocks the parent until a child ends and collects its exit status. Until the parent collects it, the finished child stays in the process table as a *zombie*.

1.2 File Descriptors, Redirection and Pipes

A file descriptor is a small integer index into the per-process table of open files. By convention 0 is standard input, 1 is standard output and 2 is standard error; `printf` simply writes to descriptor 1. The kernel always gives the *lowest free number* to a new descriptor returned by `open`, `dup` or `pipe`. The shell uses this rule for redirection: it closes 1 and opens a file, which therefore becomes descriptor 1. A `pipe` is a kernel buffer with a read end and a write end; a reader blocks while the buffer is empty and gets end-of-file only when *all* write ends are closed.

2 FORK + EXEC + WAIT

2.1 The Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        /* step 1.3: execlp("date", "date", (char *)NULL); */
        /* step 1.4: execlp("nosuchprog", "nosuchprog", (char *)NULL); */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Listing 2: run.c

The two commented lines in the child are the variants of steps 1.3 and 1.4: for each experiment I replaced the `execlp` line with one of them and compiled again.

2.2 Running It

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4211
child:  my PID is 4212, my parent is 4211
total 76
-rwxrwxr-x 1 sergiu sergiu 70984 Oct 25 19:05 run
-rw-rw-r-- 1 sergiu sergiu  1047 Oct 25 19:04 run.c
parent: child 4212 ended with status 0
```

Listing 3: Step 1.2: the child becomes ls -l

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4230
child:  my PID is 4231, my parent is 4230
```

```
Sun Oct 25 07:06:12 PM EET 2026
parent: child 4231 ended with status 0
```

Listing 4: Step 1.3: the child becomes date

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4248
child:  my PID is 4249, my parent is 4248
exec: No such file or directory
parent: child 4249 ended with status 127
```

Listing 5: Step 1.4: a program that does not exist

Observations

1. The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`?

A successful `exec` never returns: it throws away the whole program of the child – including the lines after `execlp` – and loads `ls` or `date` in its place. The code after the call simply does not exist any more in that process. With `nosuchprog`, `execlp` searched every directory of `PATH`, found nothing, returned -1 with `errno = ENOENT`, and only then the child continued with the next line: `perror` printed "No such file or directory" and `exit(127)` ended it.

2. Which number does the child print as "my parent", and whose PID is it?

The child printed `my parent is 4211`, which is exactly the number the parent printed as `my PID is 4211` on the first line: `getppid()` in the child returns the PID of the `./run` process that called `fork`. The child itself got the next free PID, 4212, which is also the value `fork` returned to the parent and the one it reports at the end.

3. What status does the parent see with `ls`, and what status with `nosuchprog`?

With `ls` (and `date`) the status is `0`: the program ran successfully and called `exit(0)`. With `nosuchprog` it is `127`, the value passed to `exit(127)` after the failed `exec`. `127` is the same convention the shell uses for "command not found" (as observed in Laboratory Work No. 3). `WEXITSTATUS` extracts this number from the `status` word filled by `waitpid`.

3 A ZOMBIE, ON PURPOSE

`zombie.c` is the original `run.c` (with `ls`) plus one line, `sleep(20);`, just before `waitpid`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* run.c with a sleep before wait: the finished child stays a zombie */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    sleep(20);                                   /* the new line */
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Listing 6: zombie.c

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null & sleep 2; ps -o pid,ppid,stat,cmd
[1] 4301
    PID    PPID STAT CMD
   3874    3866 Ss   -bash
   4301    3874 S    ./zombie
   4303    4301 Z    [ls] <defunct>
   4304    3874 R+   ps -o pid,ppid,stat,cmd
$ # ... 20 seconds later
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   3874    3866 Ss   -bash
   4322    3874 R+   ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
```

Listing 7: The process table while the parent sleeps, and 20 seconds later

Observations

1. Which letter does `ls` have in the STAT column, and what does it mean?

`ls` has the letter `Z`, and `ps` shows it as `[ls] <defunct>`: it is a *zombie*. `ls` finished its work in a few milliseconds and called `exit`, so its memory and descriptors are already freed, but its entry in the process table (PID, exit status) is kept until the parent reads the status. The parent `./zombie` itself is in state `S` (sleeping) inside `sleep(20)`.

2. After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear?

No, after 20 seconds the line is gone (and so is `./zombie`, which finished; bash reports it as `Done`). The line that removed the zombie is `waitpid(pid, &status, 0);`: when the parent woke up and called it, the kernel handed over the exit status and then deleted the child's entry from the process table. (If the parent had exited without waiting, the zombie would have been adopted and reaped by `init/systemd`.)

4 REDIRECTION: LS -L > OUT.TXT

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                               /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Listing 8: redirect.c

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 140
-rw-rw-r-- 1 sergiu sergiu     0 Oct 25 19:12 out.txt
-rwxrwxr-x 1 sergiu sergiu 71048 Oct 25 19:12 redirect
```

Listing 9: Running redirect and reading out.txt

Observations

1. Which number did open give, and why exactly that one?

`open` returned descriptor `1`. The kernel always assigns the *lowest unused* descriptor number. In the child, 0 (stdin) and 2 (stderr) were still open, and 1 had just been released by `close(1)`, so 1 was the smallest free number. The message was printed with `fprintf(stderr, ...)` on descriptor 2 precisely because descriptor 1 already pointed to the file.

2. `ls` was not changed at all. Why does its listing not appear on the screen?

`ls` always writes its output to descriptor 1 and has no idea what 1 is. `exec` replaces the program but keeps the table of open descriptors, so the `ls` that was loaded into the child inherited a descriptor 1 that points to `out.txt` instead of the terminal. That is the whole trick of redirection: it is done by the shell between `fork` and `exec`, and the program does not need any special code. (`out.txt` even appears in its own listing with size 0, because `ls` read the directory before writing anything.) The parent did not touch its own descriptor 1, so its `done:` message still went to the screen.

3. To redirect the errors instead (like `2> err.txt`), which descriptor would you close before `open`?

Descriptor 2. After `close(2)` the lowest free number is 2, so `open("err.txt", ...)` would return 2 and every error message of the program (`perror`, `fprintf(stderr, ...)`) would go to the file, while normal output would stay on the terminal.

5 A PIPE: LS /ETC | WC -L

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                /* the parent uses neither end */
    close(p[1]);                                /* step 4.3: comment this out and wc never ends */
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Listing 10: pipeline.c

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
213
$ ls /etc | wc -l
213
```

Listing 11: Comparing pipeline with the real shell

Then I broke the program on purpose by commenting out the parent's `close(p[1]);`:

```c
    close(p[0]);                                /* the parent uses neither end */
    /* close(p[1]); */
```

Listing 12: The change of step 4.3

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
                                    <- nothing is printed, the prompt does not return

(second terminal)
$ ps -o pid,ppid,stat,cmd -C pipeline,ls,wc
    PID    PPID STAT CMD
   4420    3874 S+   ./pipeline
   4422    4420 S+   wc -l

(first terminal)
^C
$
```

Listing 13: The broken pipeline hangs; ps from a second terminal

Observations

1. Did `./pipeline` and `ls /etc | wc -l` give the same number?

Yes, both printed `213`. The program does exactly what bash does for a pipeline: create the pipe, fork one child per command, connect descriptor 1 of the first and descriptor 0 of the second to the two ends with `close` + `dup`, close all the other ends and `exec`.

2. After the change, which of the two children never ends, `ls` or `wc`?

`wc` never ends. `ls` wrote its 213 names into the pipe and exited normally (it was already reaped by the first `wait`, so it does not appear in `ps`); `wc -l` stays in state `S`, blocked in `read`, and the parent stays blocked in the second `wait(NULL)` waiting for it.

3. Why, in one sentence: what is `wc` waiting for?

`wc` is waiting for end-of-file on its standard input, which the pipe delivers only when *every* write end is closed – and the parent still holds `p[1]` open, so as far as the kernel knows someone may still write, and `wc` waits forever.

6 OPTIONAL: THREE STAGES

`pipeline3.c` builds `ls /usr/bin | grep x | wc -l` with two pipes and three children. The middle child, `grep`, uses both pipes: its descriptor 0 becomes the read end of the first pipe and its descriptor 1 the write end of the second. Every child closes all four pipe ends after its `dup`s, and so does the parent; otherwise `grep` or `wc` would never see end-of-file, exactly as in the experiment above.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /usr/bin | grep x | wc -l   */

static void close_all(int a[2], int b[2])
{
    close(a[0]); close(a[1]);
    close(b[0]); close(b[1]);
}

int main(void)
{
    int p1[2], p2[2];                           /* ls -> grep, grep -> wc */
    if (pipe(p1) < 0 || pipe(p2) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: 1 becomes p1[1] */
        close(1);
        dup(p1[1]);
        close_all(p1, p2);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* grep: 0 is p1[0], 1 is p2[1] */
        close(0);
        dup(p1[0]);
        close(1);
        dup(p2[1]);
        close_all(p1, p2);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("exec grep");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: 0 becomes p2[0] */
        close(0);
        dup(p2[0]);
        close_all(p1, p2);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }
```

```
47
48      close_all(p1, p2);                          /* the parent uses no end at all */
49      wait(NULL);
50      wait(NULL);
51      wait(NULL);
52      return 0;
53  }
```

Listing 14: pipeline3.c

```
$ gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
187
$ ls /usr/bin | grep x | wc -l
187
```

Listing 15: The same number as the shell

CONCLUSIONS

In this laboratory work I reproduced, in a few lines of C, the main jobs of a Unix shell. `run.c` showed that starting a command is always the same three steps: `fork` makes a copy, `exec` turns the copy into another program and never returns on success, and `wait` collects the exit status – 0 for a successful program and 127 for one that could not be started. `zombie.c` made visible the state between the end of a child and the moment its parent waits for it: the `Z` / `<defunct>` entry that disappears only after `waitpid`.

`redirect.c` and `pipeline.c` showed that redirection and pipes do not require any cooperation from the programs: the shell only rearranges file descriptors between `fork` and `exec`, using the rule that `open` and `dup` return the lowest free number, and `exec` preserves them. Breaking the pipeline on purpose taught me the most important rule for pipes: every unused end must be closed, because a reader gets end-of-file only when all writers are gone. The optional three-stage pipeline confirmed this on a longer chain. In Laboratory Work No. 8 the same calls are used inside xv6.